

★ Chapter 8

- IR sys evaluated using a test collection:
 - 1) **Doc collection**
 - 2) **Queries / info needs**
 - 3) **Relevance judgements** [docs marked a R / NR]
- Query → what user types → simplified version of info need (what it wants)
- Gold standard / ground truth → correct relevance judgments by human experts usually
- Tuning parameters on **test collection** creates bias – overfitting
- The **ranking function** determines **how documents are ordered in response to a query**.
 - Dot product and cosine sim
 - Cosine sim preferred cause removes bias twrds longer docs bec of normalization
- **Term selection** → stemming, lemmatization, stop word removal, case folding, tokenization, normalization
- Stop word removal → remove common words → reduce noise + index size → may remove useful context sometimes
- Stemming → reduce to stem/root form → doesn't always produce a grammatically correct word → faster than lemmatization
- Lemmatization → reduce word to dictionary/base form → always valid → more accurate than stemming
- Case folding → convert to lowercase
- Normalization → remove punctuation + handling accents
- **Term weighting** → TF, TF-IDF
- TF-idf balances both local and global importance
- User only looks at top few results → so goal is to rank relevant docs as high as possible

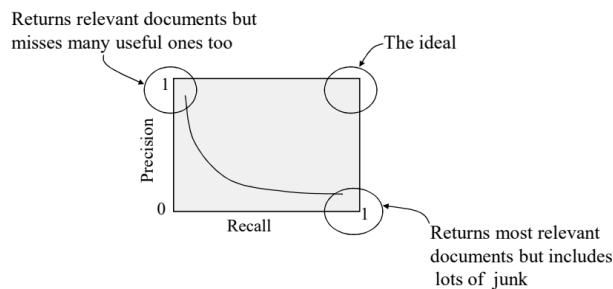
- **Difficulty in IR eval**

- Effectiveness depends on relevance
 - Relevance is continuous not binary → leads to loss of detail cuz we only consider docs as either R/NR and not partially
 - Relevance is subjective
 - context/Situation dependent
 - Cognitive nature of relevance
 - Dynamic nature of relevance → changes over time
- IR systems must try to **infer the real information need from the query**
 - Info need is detailed + in natural language + specific
 - Ad hoc IR → **retrieving documents for new, unseen queries without prior knowledge**

› Evaluation of unranked retrieval sets:

- 1) Precision
 - 2) Recall
 - 3) F-measure / f- score
 - 4) Fall out
- flat/unranked results → all retrieved docs treated equally important
 - **Precision** measures **how accurate the retrieved results are**
 - $\text{Precision} = (\text{Relevant Retrieved}) / (\text{Total Retrieved})$
 - High P ⇒ few irrelevant docs
 - **Recall** measures **how many relevant documents are found** → Focuses on completeness → *Out of all relevant documents, how many did the system retrieve?*
 - $\text{Recall} = (\text{Relevant Retrieved}) / (\text{Total Relevant in collection})$
-
- **Increasing precision** often **decreases recall**
 - **Increasing recall** often **decreases precision**

- 2 types of errors
 - **FP** → retrieves a **non-relevant doc** → **P decreases**
 - **FN** → **fails to retrieve a relevant doc** → **R decreases**
- **Precision critical tasks** → accuracy is more important than completeness → fast → focus on ranking top results correctly
- **Recall critical tasks** → **finding all relevant** information is **critical** → can be slow → no tolerance for missing docs → focus on retrieving as many Rel docs as possible
- Improving precision → reduce FPs → recall decreases
- Improving recall → retrieve more docs → may include irrelevant docs → lowers P



Precision and Recall are inverse proportional

High Precision, Low Recall (Top Left)

- **The Behavior:** The system is very "picky." It only returns results it is extremely confident about.
- **The Result:** You get zero "junk" (high precision), but you miss out on many other relevant documents that were slightly less obvious to the algorithm.

Low Precision, High Recall (Bottom Right)

- **The Behavior:** The system is "greedy;" it tries to grab every possible relevant document it can find.
- **The Result:** You find almost everything useful that exists (high recall), but you also pull in a massive amount of irrelevant "noise" or junk.

The Ideal (Top Right)

- **The Behavior:** This is the "Holy Grail" of IR.
- **The Result:** The system finds **every** relevant document (Recall = 1) and **only** relevant documents (Precision = 1).

- **F-measure** → provides balanced evaluation → ensures both accuracy n completeness considered

$$F = \frac{(\beta^2 + 1) \cdot P \cdot R}{(\beta^2 \cdot P + R)}$$

› **BETA CONTROLS** importance of P and R

- $\beta < 1$ → **Precision favoured**
- $\beta > 1$ → **Recall favoured**
- $\beta = 1$ → **equally important [F1-score]**

$$F1 = \frac{2PR}{P + R}$$

- Harmonic mean of P and R
- Gives balanced score only wehn both r high

- **Problems w P and R** →

- Dont consider tot number of irrelevant docs
- RECALL undefined if no relevant doc in collection
- PRECISION undefined when no doc retrieved

- **Fallout rate** → measures proportion of nonrelevant docs that are incorrectly retrieved → the lower the better

$$\text{Fallout} = \frac{\text{Non-relevant Retrieved}}{\text{Total Non-relevant Documents in Collection}}$$

- Precision also considers non-relevant retrieved but:
 - Precision = based only on **retrieved set**
 - Fallout = based on **entire collection**

› Evaluation of ranked retrieval sets:

- 1) Precision –Recall Curve
- 2) Average Precision
- 3) Mean Average Precision (MAP)
- 4) Kappa statistics
- 5) Cumulative Gain
- 6) Discount Cumulative Gain
- 7) Normalized Discount Cumulative Gain

- In **ranked retrieval** (like search engines), **documents** are **returned in a sorted order of relevance**
- evaluation considers: **Position of relevant documents**, not just whether they are retrieved
- Evaluation done on top k results → cuz lower ranked docs ignored by users

👉 Precision at k (P@k)

- Precision can be calculated at a specific rank k:

$$P@k = \frac{\text{Relevant documents in top } k}{k}$$

$$P@5 = \frac{3}{5} = 0.6$$

- Eg; Top 5 results → 3 are relevant
- This tells us **how accurate the top results** are

1) Precision–Recall Curve

- shows the **relationship between precision and recall at different thresholds**
- As we **move down the ranked list**:
 - **Recall increases** (more relevant docs found)
 - **Precision** usually **decreases** (more non-relevant docs included)

- 1) Go thru ranked list step by step
- 2) At each position → compute P and R
- 3) Plot ⇒ R on x , P on y

n	doc #	relevant
1	588	x
2	589	x
3	576	
4	590	x
5	986	
6	592	x
7	984	
8	988	
9	578	
10	985	
11	103	
12	591	
13	772	x
14	990	

Let total # of relevant docs = 6
Check each new recall point:

$$R=1/6=0.167; P=1/1=1$$

$$R=2/6=0.333; P=2/2=1$$

$$R=3/6=0.5; P=3/4=0.75$$

$$R=4/6=0.667; P=4/6=0.667$$

$$R=5/6=0.833; P=5/13=0.38$$

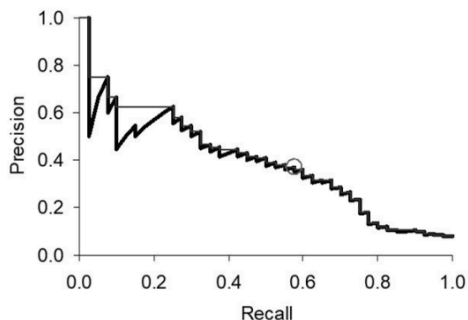
Missing one
relevant document.
Never reach
100% recall

1. **Recall (R):** $\frac{\text{Relevant docs found so far}}{\text{Total relevant in collection (6)}}$

- At $n = 1$: We found 1 relevant doc. $R = 1/6 \approx 0.167$.
- At $n = 4$: We found 3 relevant docs. $R = 3/6 = 0.50$.

2. **Precision (P):** $\frac{\text{Relevant docs found so far}}{\text{Total docs examined (rank } n \text{)}}$

- At $n = 1$: 1 relevant out of 1 seen. $P = 1/1 = 1.0$.
- At $n = 4$: 3 relevant out of 4 seen. $P = 3/4 = 0.75$.



When you plot these (R, P) pairs on a graph, you get the jagged line seen in Image 2. This is often called a **Sawtooth Curve**. Here is why it looks like that:

- **The Spikes (Up and Right):** Every time the system hits a relevant document, Recall increases (moves right) and Precision usually increases or stays high (moves up).
- **The Drops (Straight Down):** Every time the system hits a non-relevant document, Recall stays exactly the same, but Precision drops because the denominator (n) increased. This creates the vertical "teeth" in the graph.

2) Average Precision AP

- Considers:
 - **Order of documents**
 - **Positions of relevant documents**
- It computes the **average of precision values at ranks where relevant documents occur**
- Rewards sys if relevant docs placed higher

$$AP = \frac{1}{\text{Total Relevant Documents}} \sum (\text{Precision at each relevant document})$$

- We calculate precision **only at ranks where documents are relevant**
- Then take the **average of those precision values**

 = the relevant documents

Ranking #1

										
Recall	0.17	0.17	0.33	0.5	0.67	0.83	0.83	0.83	0.83	1.0
Precision	1.0	0.5	0.67	0.75	0.8	0.83	0.71	0.63	0.56	0.6

Ranking #2

										
Recall	0.0	0.17	0.17	0.17	0.33	0.5	0.67	0.67	0.83	1.0
Precision	0.0	0.5	0.33	0.25	0.4	0.5	0.57	0.5	0.56	0.6

Ranking #1: $(1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6)/6 = 0.78$

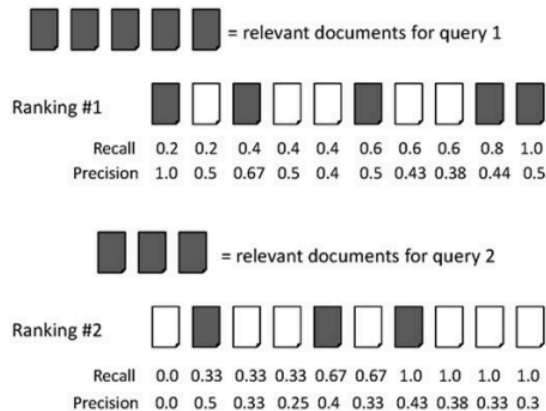
Ranking #2: $(0.5 + 0.4 + 0.5 + 0.57 + 0.56 + 0.6)/6 = 0.52$

- Higher AP → Relevant documents appear **earlier in the ranking**

3) Mean Average Precision MAP

- evaluation metric for **multiple queries**
- computes the **mean (average) of Average Precision (AP) over all queries**

- While AP evaluates performance for **one query**, MAP evaluates **overall system performance**
- Evaluates **overall system performance**
- Higher MAP: Relevant documents appear **earlier across queries**
 - 1) **For each query**: Compute **Average Precision (AP)**
 - 2) Then: Take the **average of all AP values**



$$\text{average precision query 1} = (1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$$

$$\text{average precision query 2} = (0.5 + 0.4 + 0.43)/3 = 0.44$$

$$\text{mean average precision} = (0.62 + 0.44)/2 = 0.53$$

- Missing relevant documents **directly lowers AP and MAP** → Cuz its contribution is 0
- MAP treats **each query equally** ⇒ **macroavg** ⇒ Every query contributes **one AP value**, and all are averaged equally
- MAP assumes users care about **all relevant documents** → not ideal for web search → users look only at top k
- MAP **requires knowing: Which documents are relevant for each query** → **time consuming** cuz we need manual relevance judgements

4) Kappa Statistics:

- Measures **how much two judges agree**, while **correcting** for **agreement that could happen by chance**

- $\kappa = 1$ → Perfect agreement
- $\kappa = 0$ → Agreement is just by chance
- $\kappa < 0$ → Worse than random

Table 8.2 Calculating the kappa statistic.

		judge 2 relevance		
		yes	no	total
judge 1 relevance	yes	300	20	320
	no	10	70	80
	total	310	90	400

Observed proportion of the times the judges agreed

$$P(A) = (300 + 70)/400 = 370/400 = 0.925$$

Pooled marginals

$$P(\text{nonrelevant}) = (80 + 90)/(400 + 400) = 170/800 = 0.2125$$

$$P(\text{relevant}) = (320 + 310)/(400 + 400) = 630/800 = 0.7878$$

Probability that the two judges agreed by chance

$$P(E) = P(\text{nonrelevant})^2 + P(\text{relevant})^2 = 0.2125^2 + 0.7878^2 = 0.665$$

Kappa statistic

$$\kappa = (P(A) - P(E))/(1 - P(E)) = (0.925 - 0.665)/(1 - 0.665) = 0.776$$

👉 Step 1: Observed Agreement $P(A)$

- This is the proportion of times both judges **actually** agreed

Agreement cases:

- Both said Yes → 300
- Both said No → 70

$$P(A) = \frac{300 + 70}{400} = \frac{370}{400} = 0.925$$

👉 Judges agree 92.5% of the time

👉 Step 2: Probability of Agreement by Chance $P(E)$

👉 Compute Marginal Probabilities

- Probability of non-relevant:

$$P(\text{non-relevant}) = \frac{80 + 90}{800} = \frac{170}{800} = 0.2125$$

- Probability of relevant:

$$P(\text{relevant}) = \frac{320 + 310}{800} = \frac{630}{800} = 0.7878$$

- **P(non-relevant)** → no of docs judge 1 voted irrelevant + no of docs judge 2 voted irrelevant
- **P(relevant)** → no of docs judge 1 voted relevant + no of docs judge 2 voted relevant

👉 Chance Agreement Formula

$$P(E) = P(\text{non-relevant})^2 + P(\text{relevant})^2$$

$$P(E) = (0.2125)^2 + (0.7878)^2 = 0.665$$

👉 So, 66.5% agreement could happen by chance

👉 Step 3: Kappa Formula

$$\kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

$$\kappa = \frac{0.925 - 0.665}{1 - 0.665} = \frac{0.26}{0.335} = 0.776$$

👉 Interpretation of Kappa

- $\kappa = 0.776$

👉 This indicates:

- **Strong agreement** between judges
- Agreement is **much better than chance**

5) Cumulative Gain **CG**

- **How relevant the retrieved documents are overall**

- **adding up the relevance scores** of all documents in the result list.
- Changing order of docs doesn't affect CG → doesn't consider ranking

$$CG = \sum_{i=1}^n rel_i$$

- **rel_i** = relevance score of document at position *i*
- **n** = total number of retrieved documents

6) Discounted Cumulative Gain (DCG)

- Considers both relevance and position
- measures **how good the ranking is**, giving **higher importance to top-ranked documents**.

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2 i}$$

$$DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log(1+i)}$$

- rel_i = relevance of document at position *i*
 - $\log_2 i$ = discount factor (reduces importance as position increases)
 - p = rank position up to which we calculate DCG
- As **i increases**, **log₂i increases** → **value decreases** → So lower-ranked documents contribute less

- **$2^{rel_i} - 1$ amplifies highly relevant documents**
- Example:
 - $rel = 3 \rightarrow 2^3 - 1 = 7$
 - $rel = 1 \rightarrow 2^1 - 1 = 1$

^ high relevance gets much more importance

Relevance scores:

(3, 2, 3, 0, 1, 2)

Let's compute DCG up to position 6:

$$DCG = 3 + \frac{2}{\log_2 2} + \frac{3}{\log_2 3} + \frac{0}{\log_2 4} + \frac{1}{\log_2 5} + \frac{2}{\log_2 6}$$

$$DCG \approx 3 + 2 + 1.89 + 0 + 0.43 + 0.77 = 8.09$$

- DCG values are **not normalized**
- **Hard to compare** between different queries

$$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

i	rel_i	$\log_2(i+1)$	$\frac{rel_i}{\log_2(i+1)}$
1	3	1	3
2	2	1.585	1.262
3	3	2	1.5
4	0	2.322	0
5	1	2.585	0.387
6	2	2.807	0.712

$$DCG_6 = 3 + 1.262 + 1.5 + 0 + 0.387 + 0.712$$

$$DCG_6 = 6.861$$

7) Normalized Discounted Cumulative Gain (NDCG)

- measures **how good your ranking is compared to the best possible (ideal) ranking**
- DCG → gives a score
- **NDCG** → tells **how close you are to perfect ranking**

$$NDCG_n = \frac{DCG_n}{IDCG_n}$$

Where:

- DCG_n = DCG of your current ranking
- $IDCG_n$ = DCG of the *ideal ranking* (best possible order)

› **IDCG** ⇒

- Sort docs in descending order of relevance
- Original relevance: (3, 2, 3, 0, 1, 2) ⇒ Ideal ranking: **(3, 3, 2, 2, 1, 0)**

✓ **Step 1: DCG (already computed)**

From previous: $DCG = 6.861$

✓ **Step 2: Compute IDCG**

Use same formula but with ideal order:

i	rel _i	log ₂ (i+1)	value
1	3	1	3
2	3	1.585	1.892
3	2	2	1
4	2	2.322	0.861
5	1	2.585	0.387
6	0	—	0

$$IDCG \approx 3 + 1.892 + 1 + 0.861 + 0.387 = 7.14$$

✓ **Step 3: Compute NDCG**

$$NDCG = \frac{6.861}{7.14} \approx 0.96$$

🎯 5. Interpretation

NDCG Value	Meaning
1	Perfect ranking ✅
0.9+	Very good 🔥
0.5	Moderate
< 0.3	Poor ❌

With NDCG:

- Values are always between **0 and 1**
- Makes comparison fair ✅

★ Normalized DCG (Example)

4 documents: d_1, d_2, d_3, d_4

i	Ground Truth		Ranking Function ₁		Ranking Function ₂	
	Document Order	r_i	Document Order	r_i	Document Order	r_i
1	d4	2	d3	2	d3	2
2	d3	2	d4	2	d2	1
3	d2	1	d2	1	d4	2
4	d1	0	d1	0	d1	0
	NDCG _{GT} =1.00		NDCG _{RF1} =1.00		NDCG _{RF2} =0.9203	

› System Quality (Beyond Retrieval Accuracy)

- **System Quality** = **How well the system performs overall**
- **Factors:**

- Indexing speed → **How fast** the system **processes** and **stores documents** → Measured in docs per hr
- Search speed (latency) → **How fast results** are **returned after** a **query** → measured in ms
- Query language expressiveness ⇒ **How powerful** the **search queries can be**
- Performance on complex queries
- Collection size and coverage

› User Utility (User Satisfaction)

- **Affected by:**
 - ✓ **Relevance** (correct results)
 - ✓ **Speed** (fast results)
 - ✓ **Interface** (easy to use)
- Measured by :  **Return Rate:** Users coming back again

› Evaluation of System Changes (A/B Testing)

- **When improving** a **system**, we must **test changes** safely.
- **Compare two versions** of a system
 - **A = current system (control)**
 - **B = new system (variant)**
- **90–99%** users → **old** system
- **1–10%** users → **new** system

👉 Then **compare performance**

- Change one thing at a time → To **know exactly what caused improvement**

› Search Snippets

- **Short previews** shown **under** each **search result**

- Help users decide:
 - Which result to click
 - Whether it's relevant
 - **Static snippets** $\Rightarrow$ same summary for every query $\rightarrow$ pre-generated $\rightarrow$ eg; first few lines of a doc
 - **Dynamic snippets** $\Rightarrow$ **Generated based** on the **query** $\rightarrow$ **Highlight relevant parts**
 - **KWIC** [keyword in context] $\Rightarrow$ Shows **keyword (frm query)** with **surrounding text**
-

3. Consider an information need for which there are 4 relevant documents in the collection. Contrast two systems run on this collection. Their top 10 results are judged for relevance as follows (the leftmost item is the top ranked search result):

System 1: R N R N N N N R R

System 2: N R N N R R R N N N

(a) What is the MAP of each system? Which has a higher MAP?

Answer:

Relevant documents = 4

System 1: R N R N N N N R R

Precision at relevant positions:

- Rank 1 $\rightarrow 1/1 = 1$
- Rank 3 $\rightarrow 2/3 \approx 0.67$
- Rank 9 $\rightarrow 3/9 \approx 0.33$
- Rank 10 $\rightarrow 4/10 = 0.4$

$$\text{MAP}_1 = (1 + 0.67 + 0.33 + 0.4) / 4 \approx \mathbf{0.60}$$

System 2: N R N N R R R N N N

Precision at relevant positions:

- Rank 2 $\rightarrow 1/2 = 0.5$
- Rank 5 $\rightarrow 2/5 = 0.4$
- Rank 6 $\rightarrow 3/6 = 0.5$
- Rank 7 $\rightarrow 4/7 \approx 0.57$

$$\text{MAP}_2 \approx (0.5 + 0.4 + 0.5 + 0.57) / 4 \approx \mathbf{0.49}$$

👉 System 1 has higher MAP

(c) What is the R-precision of each system?

Answer:

R-Precision = Precision at rank R

$$R\text{-Precision} = \frac{\text{Number of relevant documents in top } R \text{ results}}{R}$$

R = 4 $\rightarrow$ look at top 4 results

- System 1: R N R N $\rightarrow$ 2 relevant $\rightarrow$ R-precision = $2/4 = 0.5$
- System 2: N R N N $\rightarrow$ 1 relevant $\rightarrow$ R-precision = $1/4 = 0.25$

👉 System 1 is better in both MAP and R-precision